



使用通道读写文件

北京理工大学计算机学院
金旭亮

使用通道写数据到文件-1

不能用new关键字直接实例化通道，而是通过IO流的getChannel()方法获取其引用：

```
private static void writeToFileUseBuffer() throws IOException {  
    try (var outputStream = new FileOutputStream("temp.data")) {  
        //获取文件通道  
        var channel : FileChannel = outputStream.getChannel();  
        //通道类型: class sun.nio.ch.FileChannelImpl  
        System.out.println("通道类型:" + channel.getClass());  
        //写入字符串  
        writeStringToFile(channel);  
        //写入整数  
        writeTwoIntegerToFile(channel);  
    }  
}
```

使用通道写数据到文件-2

要写入通道的数据，需要先放到Buffer中：

```
private static void writeStringToFile(FileChannel channel)
    throws IOException {
    var stringToWrite = "你好, 中国! ";
    //创建Buffer
    var stringData : byte[] = stringToWrite.getBytes(StandardCharsets.UTF_8);
    var byteBuffer : ByteBuffer = ByteBuffer.wrap(stringData);
    //将Buffer中的内容通过通道写入文件
    int numWrites = channel.write(byteBuffer);
    System.out.println("写入字符串:" + stringToWrite);
    System.out.println("写入字节数: " + numWrites);
}
```



通道的读和写，都需要通过Buffer。

使用通道写数据到文件-3

```
private static void writeTwoIntegerToFile(FileChannel channel)
    throws IOException {
    //创建一个可以保存两个整数的Buffer
    var intBuffer : ByteBuffer = ByteBuffer.allocate(Integer.BYTES * 2);
    intBuffer.putInt(100);
    intBuffer.putInt(255);
    //如果这时再次向Buffer中写入数据, 由于数据已满,
    //会抛出异常: BufferOverflowException

    //必须将读写指针移到开头, 否则, 写入的字节为0.
    intBuffer.flip();
    int numWrites = channel.write(intBuffer);
    System.out.println("写入两个整数:100 255");
    //两个整数被追加到了文件尾部
    System.out.println("写入字节数: " + numWrites);
}
```

Buffer是有大小的, 因此, 如果程序中对Buffer进行了存取操作, 则要注意Buffer的读写指针位置。

强制刷新到磁盘



在将缓冲区写入文件通道时，出于性能的原因，操作系统不会每次都实时地将数据写入到磁盘文件，而是先将其保存于系统缓冲区中。



在将缓冲区数据写入通道时，要保证数据能写入磁盘，可以在写入后调用一下FileChannel的force()方法。

使用通道从文件中读



```
private static void readFromFileUseBuffer() throws IOException {  
    //以“同步数据更新的读写”模式打开文件  
    try (var randomAccessFile = new RandomAccessFile( name: "temp.data", mode: "r");  
        var channel : FileChannel = randomAccessFile.getChannel();  
    ) {  
        //缓冲区要足够大,以便一次读入所有数据  
        byte[] content = new byte[(int) randomAccessFile.length()];  
        var buffer : ByteBuffer = ByteBuffer.wrap(content);  
        int readCount = channel.read(buffer);  
        System.out.println("读入字节数: " + readCount);  
        //读入数据的前半段是字符串  
        int stringLength = readCount - Integer.BYTES * 2;  
        var stringContent = new String(Arrays.copyOf(content, stringLength),  
                                         StandardCharsets.UTF_8);  
        System.out.println(stringContent); //输出字符串  
        //指针移到后部,读取两个整数  
        buffer.position(readCount - Integer.BYTES * 2);  
        //输出两个整数  
        System.out.println(buffer.getInt());  
        System.out.println(buffer.getInt());  
    }  
}
```

在读取数据时，要依据数据类型进行解码

使用通道复制文件



通道提供了transferTo或transferFrom两个方法，可以在两个通道之间高速地传输数据：

```
private static void copyFileUseChannel() throws IOException {  
    try (var randomAccessFile = new RandomAccessFile( name: "data.txt", mode: "rwd");  
         var channel : FileChannel = randomAccessFile.getChannel();  
         var targetFile = new RandomAccessFile( name: "data_copied.txt", mode: "rw");  
         var targetChannel : FileChannel = targetFile.getChannel()) {  
        //以下两种方式，都可以实现文件复制  
        //var numberTransferred=targetChannel.transferFrom(channel,0,channel.size());  
        var numberTransferred : long = channel.transferTo( position: 0,  
                                                         channel.size(), targetChannel);  
        System.out.println("共复制字节: " + numberTransferred);  
        System.out.println(Files.readString(Paths.get( first: "data_copied.txt")));  
    }  
}
```


文件复制性能大比拼



四种文件复制方式

一对一的字节复制

带缓冲区的字节复制

使用NIO的Channel和
Buffer

使用NIO的Channel和
Transfer方法

一对一的字节复制

```
FileCopyRunner noBufferStreamCopy = new FileCopyRunner() {  
    1 usage  
    @Override  
    public void copyFile(File source, File target) {  
        try (var fin = new FileInputStream(source);  
            var fout = new FileOutputStream(target);) {  
            int result;  
            while ((result = fin.read()) != -1) {  
                fout.write(result);  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}  
  
@Override  
public String toString() { return "noBufferStreamCopy"; }  
};
```

使用缓冲区的文件复制

```
FileCopyRunner bufferedStreamCopy = new FileCopyRunner() {  
    1 usage  
    @Override  
    public void copyFile(File source, File target) {  
        try (var fin = new BufferedInputStream(new FileInputStream(source));  
             var fout = new BufferedOutputStream(new FileOutputStream(target));) {  
            byte[] buffer = new byte[1024];  
            int result;  
            while ((result = fin.read(buffer, off: 0, buffer.length)) != -1) {  
                fout.write(buffer, off: 0, result);  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
    @Override  
    public String toString() { return "bufferedStreamCopy"; }  
};
```

使用Channel和Buffer的文件复制

```
FileCopyRunner nioBufferCopy = new FileCopyRunner() {  
    1 usage  
    @Override  
    public void copyFile(File source, File target) {  
        try {  
            var inputStream = new FileInputStream(source);  
            var fin : FileChannel = inputStream.getChannel();  
            var outputStream = new FileOutputStream(target);  
            var fout : FileChannel = outputStream.getChannel();  
        } {  
            ByteBuffer buffer = ByteBuffer.allocate( capacity: 1024);  
            while (fin.read(buffer) != -1) {  
                buffer.flip();  
                while (buffer.hasRemaining()) {  
                    fout.write(buffer);  
                }  
                buffer.clear();  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
    @Override  
    public String toString() { return "nioBufferCopy"; }  
};
```

使用Channel.transferTo()方法复制文件

```
FileCopyRunner nioTransferCopy = new FileCopyRunner() {  
    1 usage  
    @Override  
    public void copyFile(File source, File target) {  
        try {  
            var inputStream = new FileInputStream(source);  
            var fin : FileChannel = inputStream.getChannel();  
            var outputStream = new FileOutputStream(target);  
            var fout : FileChannel = outputStream.getChannel();  
        } {  
            long transferred = 0L;  
            long size = fin.size();  
            while (transferred != size) {  
                transferred += fin.transferTo(position: 0, size, fout);  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
    @Override  
    public String toString() { return "nioTransferCopy"; }  
};
```

测试用代码-1

```
public class FileCopy {  
    2 usages  
    private static final int ROUNDS = 5;  
    //性能测试  
    7 usages  
    private static void benchmark(FileCopyRunner test, File source, File target) {  
        long elapsed = 0L;  
        for (int i = 0; i < ROUNDS; i++) {  
            long startTime = System.currentTimeMillis();  
            test.copyFile(source, target);  
            elapsed += System.currentTimeMillis() - startTime;  
            target.delete();  
        }  
        System.out.println(test + ": " + elapsed / ROUNDS);  
    }  
  
    public static void main(String[] args) {...}  
}
```

测试用代码-2

//大约几百K

```
File smallFile = new File( pathname: "d:/test.pptx");
File smallFileCopy = new File( pathname: "d:/test_copy.pptx");
System.out.println("---Copying small file---");
benchmark(noBufferStreamCopy, smallFile, smallFileCopy);
benchmark(bufferedStreamCopy, smallFile, smallFileCopy);
benchmark(nioBufferCopy, smallFile, smallFileCopy);
benchmark(nioTransferCopy, smallFile, smallFileCopy);
```

//大约几百M

```
File bigFile = new File( pathname: "D:/test.mp4");
File bigFileCopy = new File( pathname: "D:/test_copy.mp4");
System.out.println("---Copying big file---");
//benchmark(noBufferStreamCopy, bigFile, bigFileCopy);
benchmark(bufferedStreamCopy, bigFile, bigFileCopy);
benchmark(nioBufferCopy, bigFile, bigFileCopy);
benchmark(nioTransferCopy, bigFile, bigFileCopy);
```



```
Run: FileCopy x
"C:\Program Files\Java\jdk-17\bin\java.exe"
---Copying small file---
noBufferStreamCopy: 531
bufferedStreamCopy: 0
nioBufferCopy: 2
nioTransferCopy: 0
---Copying big file---
bufferedStreamCopy: 110
nioBufferCopy: 350
nioTransferCopy: 64

Process finished with exit code 0
```